project membership in its database, while the engineering group hard-coded department checks into the deployment tool. None of these methods was easy to audit or modify. When interns unexpectedly gained access to financial forecasts, the root cause wasn't just a poor policy; it was the absence of a central, consistent way to manage policies.

To address this problem, ACME adopted a policy-as-code approach. They didn't just "put policies in Git." Instead, they established policy repositories as the authoritative source for writing and reviewing Cedar policies, and they used policy stores as the places their authorizer queried for runtime decisions. Usually, they maintained a one-to-one mapping—one repository for one store—keeping ownership, reviews, and deployments simple. However, they treated the two as distinct: repositories are for authoring and history, while stores are for execution and operations.

- For Customer Collaboration, ACME maintained a `collab-policy` repository (including Cedar policies, schema, and tests) and a matching policy store that the app queried during runtime.
- For deployments, they maintained a `deploy-policy` repository and a `Deploy` store managed by the engineering team.
- Company-wide guardrails were stored in a `global-guardrails` repo and a central `Guardrails` store managed by the identity team.

All three followed the same deployment process: *edit in repo → validate and test with continuous integration → build a signed policy bundle → promote to the corresponding store(s) → observe decisions and roll back if needed*.